YZ 203 · VISUAL PROGRAMMING

Week 5

Keyboard, Mouse & Collision: giving the basket a player

Erciyes University

Java Swing & Event-Driven Programming

Murat AKPULAT, PhD

What's on today

The player enters the game. Input becomes a third job next to update and render.

01

Keyboard

KeyListener, key codes, the focus problem

02

Adapters and lambdas

Why KeyListener has no lambda, and KeyAdapter

03

Smooth movement

Flags set by keys, applied by the Timer

04

Key bindings

The focus-proof alternative

05

Mouse

Click, move, and drag events

06

Collision and Falling Egg v2

Rectangle.intersects, score, a playable game

Quick Recap: Where We Left Off

Week 3

Timer fires every N ms. Its listener moves things.

Week 4

We draw in paintComponent.
The Timer updates numbers and calls repaint().

Falling Egg v1

FieldPanel, Egg that draws itself, a Pause button and a speed slider.

What is missing

The player. Nothing in the game reacts to the keyboard or the mouse, and nothing can be caught.

Today the user becomes the source of events again, and the Timer has to cooperate with them.

Three Jobs in a Game Panel

Week 4 had two jobs. The player adds a third, and keeping them apart is the whole trick.

1

Input

- key and mouse listeners
- only record what the player wants
- left = true, mouseX = 120

→

2

Update

- the Timer listener
- reads the recorded input
- moves the basket, moves the egg, checks the catch

→

3

Render

- paintComponent
- draws the current state
- never changes it

Listeners write down the wish. The Timer carries it out on a fixed beat. paintComponent only draws.

Keyboard Events: Three Methods

A KeyListener is the interface for keyboard input. It has three methods, not one.

method	called when...	good for
keyPressed(KeyEvent e)	a key goes down	movement, shortcuts (the one we use most)
keyReleased(KeyEvent e)	a key comes up	stopping a movement
keyTyped(KeyEvent e)	a character is produced	text input; ignores arrows and Shift

Holding a key does not mean one keyPressed. The operating system waits a moment, then repeats it. This is why arrow-key movement feels jerky, and we will fix that.

Your First KeyListener: Key Echo

Scenario: the panel shows the name of the last key pressed. The two empty methods are required.

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class KeyPanel extends JPanel {
    private String text = "Press a key";

    KeyPanel() {
        setFocusable(true);
        addKeyListener(new KeyListener() {
            public void keyPressed(KeyEvent e) {
                text = "Pressed: " + KeyEvent.getKeyText(e.getKeyCode());
                repaint();
            }
            public void keyReleased(KeyEvent e) { }
            public void keyTyped(KeyEvent e) { }
        });
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setFont(new Font("Arial", Font.BOLD, 22));
        g.drawString(text, 20, 50);
    }
}

public class KeyEcho {
    public static void main(String[] args) {
        JFrame f = new JFrame("Key Echo");
        KeyPanel p = new KeyPanel();
        f.add(p);
        f.setSize(300, 140);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        p.requestFocusInWindow();
    }
}
```

OUTPUT

start



after Space



STEP BY STEP — WHERE OOP SHOWS UP

1. Interface — KeyListener has three methods; a class that implements it must supply all three.

2. Anonymous class — new KeyListener() { ... } is a class without a name, created on the spot.

3. Callback — we never call keyPressed. Swing calls it when a key goes down on a focused component.

Predict first: What does the panel show when you press Shift, or an arrow key, or F1?

Which Key? getKeyCode() vs getKeyChar()

Games ask "which key", not "which letter". Use the key code.

	e.getKeyCode()	e.getKeyChar()
Returns	an int constant such as KeyEvent.VK_LEFT	a char such as 'a'
Arrow keys, Space, Enter	works	no useful character
Shift or caps lock	same code for a and A	a different char
Best for	movement and controls	typing text

```
if (e.getKeyCode() == KeyEvent.VK_LEFT)  { ... }
if (e.getKeyCode() == KeyEvent.VK_SPACE) { ... }
```

OOP 1: Why Is There No Lambda for KeyListener?

A lambda stands in for **one** method. Which method would it be here?

ActionListener: one method

```
interface ActionListener {
    void actionPerformed(ActionEvent e);
}

button.addActionListener(
    e -> label.setText("Hi"));    // OK
```

The compiler knows the lambda is actionPerformed.

KeyListener: three methods

```
interface KeyListener {
    void keyPressed(KeyEvent e);
    void keyReleased(KeyEvent e);
    void keyTyped(KeyEvent e);
}

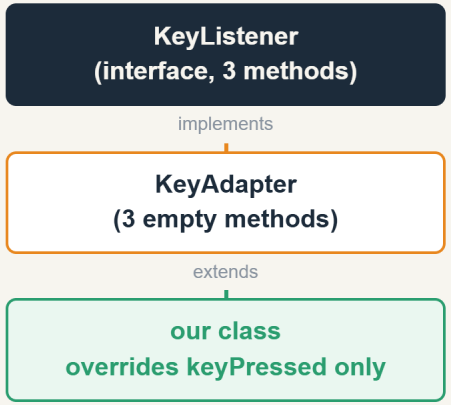
addKeyListener(e -> ...);    // ERROR
```

Three candidates. A lambda cannot choose, so it is not allowed.

An interface with exactly one abstract method is called a functional interface. Only those accept a lambda.

OOP 2: Adapter Classes Fill the Gaps

Java ships a class that already implements all three methods with empty bodies. We extend it and override only what we need.




```
addKeyListener(new KeyAdapter() {
    @Override
    public void keyPressed(KeyEvent e) {
        if (e.getKeyCode() == KeyEvent.VK_LEFT)
            left = true;
    }
});
```

Inheritance and an interface working together. MouseAdapter and MouseMotionAdapter follow the same idea.

OOP 3: Anatomy of an Anonymous Class

The lambda's older relative. It creates a subclass and one object of it in a single expression.

```
addKeyListener(new KeyAdapter() {
    @Override
    public void keyPressed(KeyEvent e) {
        left = true;
    }
});
```

part	meaning
new KeyAdapter() { ... }	create an object of a nameless subclass of KeyAdapter
@Override keyPressed	the one method we replace
(...) around it	the whole object is the argument of addKeyListener

Lambda, Anonymous Class, Adapter: When to Use Which

listener	methods	write it as
ActionListener	1	a lambda: e -> ...
ChangeListener	1	a lambda: e -> ...
KeyListener	3	new KeyAdapter() { one override }
MouseListener	5	new MouseAdapter() { the ones you need }
MouseMotionListener	2	new MouseMotionAdapter() { ... }

Rule of thumb: count the interface's methods. One means lambda. More than one means an adapter with the methods you need.

Arrow Keys, the Simple Way (and Its Problem)

Scenario: a square moves 10 pixels on each keyPressed. It works, but it does not feel like a game.

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

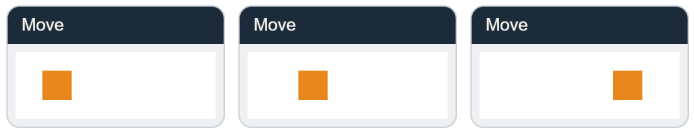
class MovePanel extends JPanel {
    private int x = 100;

    MovePanel() {
        setFocusable(true);
        addKeyListener(new KeyAdapter() {
            @Override
            public void keyPressed(KeyEvent e) {
                if (e.getKeyCode() == KeyEvent.VK_LEFT)  x -= 10;
                if (e.getKeyCode() == KeyEvent.VK_RIGHT) x += 10;
                repaint();
            }
        });
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.ORANGE);
        g.fillRect(x, 80, 40, 40);
    }
}

public class MoveJerky {
    public static void main(String[] args) {
        JFrame f = new JFrame("Arrow Keys");
        MovePanel p = new MovePanel();
        f.add(p);
        f.setSize(360, 200);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        p.requestFocusInWindow();
    }
}
```

OUTPUT



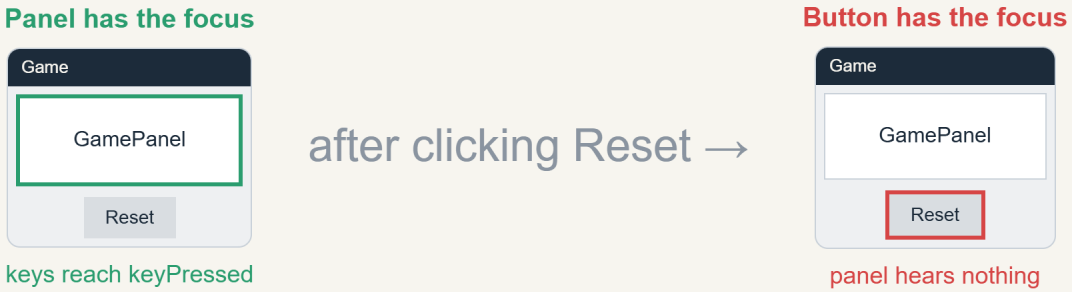
STEP BY STEP — WHERE OOP SHOWS UP

1. **Adapter —** KeyAdapter lets us write only keyPressed.
2. **The problem —** holding a key gives one step, a pause, then the OS repeat rate. Never smooth.
3. **Next —** stop moving on key events. Record the key, move on a Timer beat.

Predict first: You hold LEFT for two seconds. Is the speed constant? Where is the pause?

The Focus Problem: "I Press Keys and Nothing Happens"

Key events go to exactly one component: the one that has the keyboard focus.



tool	does
setFocusable(true)	allows a panel to take the focus (panels do not by default)
requestFocusInWindow()	asks for the focus now: call it after the window is visible
button.setFocusable(false)	a button that never steals the focus

Reproducing and Fixing the Focus Problem

Scenario: a Reset button next to the game. Delete the FIX line and click Reset: the keys stop working.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class FocusPanel extends JPanel {
    private int x = 100;

    FocusPanel() {
        setFocusable(true);
        addKeyListener(new KeyAdapter() {
            @Override
            public void keyPressed(KeyEvent e) {
                if (e.getKeyCode() == KeyEvent.VK_LEFT)  x -= 10;
                if (e.getKeyCode() == KeyEvent.VK_RIGHT) x += 10;
                repaint();
            }
        });
    }

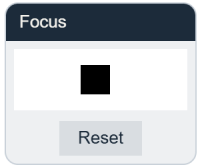
    void reset() { x = 100; repaint(); }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.fillRect(x, 60, 40, 40);
    }
}

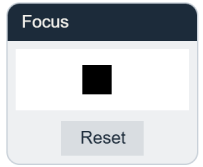
public class FocusDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Focus");
        FocusPanel p = new FocusPanel();
        JButton reset = new JButton("Reset");
        reset.setFocusable(false);   // FIX: the button never steals focus
        reset.addActionListener(e -> p.reset());
        f.add(p);
        f.add(reset, BorderLayout.SOUTH);
        f.setSize(320, 220);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        p.requestFocusInWindow();
    }
}
```

OUTPUT

keys move the square



Reset clicked, with the FIX



STEP BY STEP — WHERE OOP SHOWS UP

1. Cause — a clicked button becomes the focus owner, so key events go to it, not to the panel.

2. Fix A — setFocusable(false) on every control that should not take keys (shown).

3. Fix B — call panel.requestFocusInWindow() at the end of the button's listener.

Predict first: Comment out the FIX line, click Reset, then press the arrows. Do they still move the square?

A Stuck Key: When the Window Loses Focus

The key goes down in our window, but comes up somewhere else.

hold LEFT
left = true

→

Alt+Tab to another window

→

release LEFT
our panel never hears it

→

come back:
left is still true

```
addFocusListener(new FocusAdapter() {
    @Override
    public void focusLost(FocusEvent e) {
        left = false;
        right = false;
    }
});
```

Rule: when input can be lost, reset the recorded input.

Key bindings do not make this go away by themselves: the "released" event can also be missed.

Smooth Movement: Record, Then Apply on the Beat

Key events say "went down" and "came up". The Timer says "now move". Three small pieces.



The movement speed is now decided by the Timer, not by the keyboard repeat rate. It no longer depends on the operating system's key-repeat rate. (Swing timers are not exact, so the speed can still vary a little when the computer is busy.)

Smooth Arrow Keys: Flags and a Timer

Scenario: hold LEFT and the square glides at a constant speed; release and it stops at once.

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

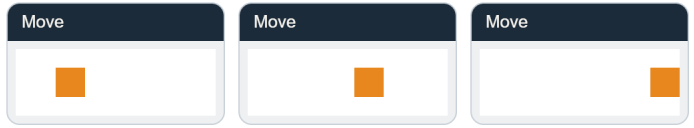
class SmoothPanel extends JPanel {
    private int x = 100;
    private boolean left, right;

    SmoothPanel() {
        setFocusable(true);
        addKeyListener(new KeyAdapter() {
            @Override
            public void keyPressed(KeyEvent e) {
                if (e.getKeyCode() == KeyEvent.VK_LEFT)  left = true;
                if (e.getKeyCode() == KeyEvent.VK_RIGHT) right = true;
            }
            @Override
            public void keyReleased(KeyEvent e) {
                if (e.getKeyCode() == KeyEvent.VK_LEFT)  left = false;
                if (e.getKeyCode() == KeyEvent.VK_RIGHT) right = false;
            }
        });
        new Timer(16, e -> {
            if (left)  x -= 5;
            if (right) x += 5;
            x = Math.max(0, Math.min(x, getWidth() - 40));
            repaint();
        }).start();
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.ORANGE);
        g.fillRect(x, 80, 40, 40);
    }
}

public class MoveSmooth {
    public static void main(String[] args) {
        JFrame f = new JFrame("Smooth Move");
        SmoothPanel p = new SmoothPanel();
        f.add(p);
        f.setSize(360, 200);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        p.requestFocusInWindow();
    }
}
```

OUTPUT



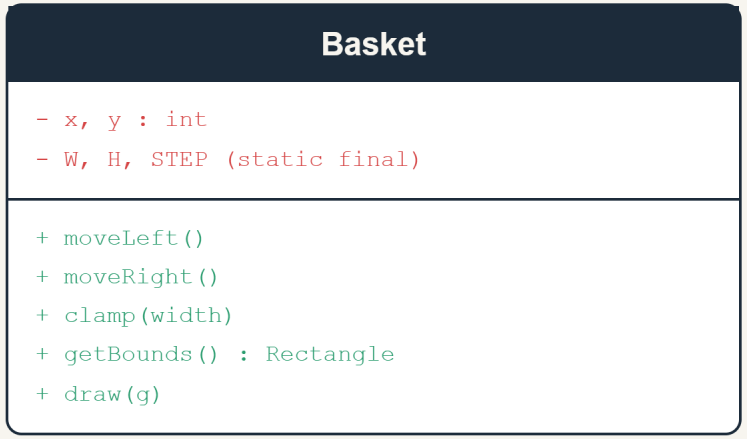
STEP BY STEP — WHERE OOP SHOWS UP

1. **Two listeners, one Timer —** KeyAdapter sets booleans, Timer reads them: the three jobs of the game panel.
2. **Clamping —** Math.max(0, Math.min(x, width - 40)) keeps the square inside.
3. **State —** left, right and x are private fields; nothing else touches them.

Predict first: What happens when LEFT and RIGHT are held together? What if we delete keyReleased()?

OOP 4: The Basket Is an Object Too

Same recipe as Egg: private state, a few public methods, and a draw(Graphics).




```
NEEDS: java.awt.*
class Basket {
    private int x;
    private final int y;
    private static final int W = 90, H = 30, STEP = 6;

    Basket(int x, int y) { this.x = x; this.y = y; }

    void moveLeft()  { x -= STEP; }
    void moveRight() { x += STEP; }

    void clamp(int fieldWidth) {
        x = Math.max(0, Math.min(x, fieldWidth - W));
    }

    Rectangle getBounds() { return new Rectangle(x, y, W, H); }

    void draw(Graphics g) {
        g.setColor(new Color(139, 90, 43));
        g.fillArc(x, y - H, W, 2 * H, 180, 180);
    }
}
```

static final constants (W, H, STEP) are shared by every Basket: this is the right use of static. **getBounds()** will be the key to collision.

Moving the Basket with the Arrow Keys

Scenario: the first playable piece. The panel owns a Basket and a Timer; keys only set flags.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class BasketPanel extends JPanel {
    private final Basket basket = new Basket(150, 300);
    private boolean left, right;

    BasketPanel() {
        setPreferredSize(new Dimension(400, 340));
        setFocusable(true);
        addKeyListener(new KeyAdapter() {
            @Override public void keyPressed(KeyEvent e)  { setKey(e.getKeyCode(), true); }
            @Override public void keyReleased(KeyEvent e) { setKey(e.getKeyCode(), false); }
        });
        new Timer(16, e -> {
            if (left)  basket.moveLeft();
            if (right) basket.moveRight();
            basket.clamp(getWidth());
            repaint();
        }).start();
    }

    private void setKey(int code, boolean down) {
        if (code == KeyEvent.VK_LEFT)  left = down;
        if (code == KeyEvent.VK_RIGHT) right = down;
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        basket.draw(g);
    }
}
// plus the Basket class from the previous slide in the same file

public class BasketKeys {
    public static void main(String[] args) {
        JFrame f = new JFrame("Basket");
        BasketPanel p = new BasketPanel();
        f.add(p);
        f.setSize(400, 380);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        p.requestFocusInWindow();
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

1. **Composition —** BasketPanel has a Basket; it asks it to move, clamp and draw.
2. **Helper method —** setKey() removes the duplicated if-lines from both listeners.
3. **Add the Basket class —** paste it from the previous slide into the same file.

Predict first: What happens if we remove basket.clamp(getWidth())?

Key Bindings: Keys That Ignore Focus

Instead of listening for keys, we map a keystroke to a named action. Swing finds it even when another component has the focus, as long as the window is active.



Two tables: keystroke to name, name to action. The name in the middle makes the mapping easy to change later. Caution: a slider, a text field or a combo box consumes arrow keys itself, so such a control still wins while it has the focus.

The Same Square with Key Bindings

Scenario: no setFocusable, no requestFocusInWindow. It keeps working after clicking a button.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class BindPanel extends JPanel {
    private int x = 100;

    BindPanel() {
        InputMap im = getInputMap(WHEN_IN_FOCUSED_WINDOW);
        ActionMap am = getActionMap();
        im.put(KeyStroke.getKeyStroke("LEFT"), "left");
        im.put(KeyStroke.getKeyStroke("RIGHT"), "right");
        am.put("left", new AbstractAction() {
            public void actionPerformed(ActionEvent e) { x -= 10; repaint(); }
        });
        am.put("right", new AbstractAction() {
            public void actionPerformed(ActionEvent e) { x += 10; repaint(); }
        });
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.ORANGE);
        g.fillRect(x, 80, 40, 40);
    }
}

public class MoveBindings {
    public static void main(String[] args) {
        JFrame f = new JFrame("Key Bindings");
        BindPanel p = new BindPanel();
        f.add(p);
        f.setSize(360, 200);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

1. Interface again — AbstractAction implements Action, which extends ActionListener: the Week 1 idea.

2. WHEN_IN_FOCUSED_WINDOW — the binding works while the window is active, even if a button has the focus. Sliders and text fields may still consume arrow keys.

3. For smooth movement — see the next slide: bind "pressed LEFT" and "released LEFT" to set and clear a flag.

Key Bindings, Smooth: the Same Flags and Timer

Scenario: the problem from the first bindings example is gone. Same smooth glide as the KeyListener version.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class BindSmoothPanel extends JPanel {
    private int x = 100;
    private boolean left, right;

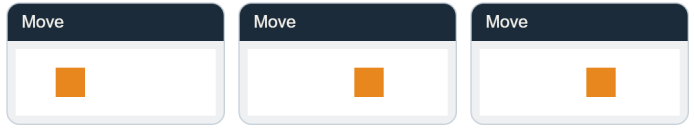
    BindSmoothPanel() {
        InputMap im = getInputMap(WHEN_IN_FOCUSED_WINDOW);
        ActionMap am = getActionMap();
        bind(im, am, "pressed LEFT",   "leftOn",   () -> left = true);
        bind(im, am, "released LEFT",  "leftOff",  () -> left = false);
        bind(im, am, "pressed RIGHT",  "rightOn",  () -> right = true);
        bind(im, am, "released RIGHT", "rightOff", () -> right = false);
        new Timer(16, e -> {
            if (left)  x -= 5;
            if (right) x += 5;
            x = Math.max(0, Math.min(x, getWidth() - 40));
            repaint();
        }).start();
    }

    private void bind(InputMap im, ActionMap am, String key,
                      String name, Runnable r) {
        im.put(KeyStroke.getKeyStroke(key), name);
        am.put(name, new AbstractAction() {
            public void actionPerformed(ActionEvent e) { r.run(); }
        });
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.ORANGE);
        g.fillRect(x, 80, 40, 40);
    }
}

public class MoveBindingsSmooth {
    public static void main(String[] args) {
        JFrame f = new JFrame("Bindings, Smooth");
        BindSmoothPanel p = new BindSmoothPanel();
        f.add(p);
        f.setSize(360, 200);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

1. Same design — the actions only set flags; the Timer moves. Compare with the KeyListener version slide by slide.

2. "pressed" and "released" — KeyStroke strings that bind the key going down and coming up separately.

3. Runnable as a parameter — () -> left = true is a lambda for Runnable, so one helper method does all four bindings.

KeyListener vs Key Bindings

	KeyListener	Key bindings
Needs the focus	yes: setFocusable plus requestFocus	much less: works while the window is active
Breaks when a button is clicked	yes, unless fixed	no; but sliders and text fields still take their own arrow keys
Setup	short: one adapter	longer: InputMap, ActionMap, Action
Good for	small demos and learning	real applications and projects
Event objects	KeyEvent with key code	just an ActionEvent

We use KeyListener in the labs because it is quick to read. Remember bindings for the semester project, where focus problems cost the most time.

Mouse Events: Two Interfaces and Their Adapters

interface	methods	adapter
MouseListener	mousePressed, mouseReleased, mouseClicked, mouseEntered, mouseExited	MouseAdapter
MouseMotionListener	mouseMoved (no button), mouseDragged (button down)	MouseMotionAdapter

you need	in the MouseEvent e
where?	e.getX(), e.getY(): relative to the component that has the listener
which button?	SwingUtilities.isLeftMouseButton(e), isRightMouseButton(e)
no focus needed	mouse events go to the component under the pointer

A Circle That Follows the Mouse

Scenario: the circle is centered on the pointer. mouseMoved fires for every small move.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class FollowPanel extends JPanel {
    private int mx = 100, my = 60;

    FollowPanel() {
        addMouseMotionListener(new MouseMotionAdapter() {
            @Override
            public void mouseMoved(MouseEvent e) {
                mx = e.getX();
                my = e.getY();
                repaint();
            }
        });
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.ORANGE);
        g.fillOval(mx - 15, my - 15, 30, 30);
    }
}

public class MouseFollow {
    public static void main(String[] args) {
        JFrame f = new JFrame("Mouse Follow");
        FollowPanel p = new FollowPanel();
        f.add(p);
        f.setSize(300, 200);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        p.requestFocusInWindow();
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

1. **MouseMotionAdapter —** two methods in the interface, we override one.
2. **Centering —** x - 15 and y - 15 turn "center" into the top-left corner fillOval expects.
3. **Same pattern —** the listener only records mx and my; paintComponent draws.

Predict first: Why do we subtract 15 from mx and my before drawing the circle?

Clicks: Position, Button and a Counter

Scenario: left click drops an orange circle, right click a blue one, and the panel counts clicks.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class ClickPanel extends JPanel {
    private int x = 100, y = 60, clicks = 0;
    private Color color = Color.ORANGE;

    ClickPanel() {
        addMouseListener(new MouseAdapter() {
            @Override
            public void mousePressed(MouseEvent e) {
                x = e.getX();
                y = e.getY();
                clicks++;
                color = SwingUtilities.isRightMouseButton(e)
                    ? Color.BLUE : Color.ORANGE;
                repaint();
            }
        });
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(color);
        g.fillOval(x - 20, y - 20, 40, 40);
        g.setColor(Color.DARK_GRAY);
        g.drawString("Clicks: " + clicks, 10, 20);
    }
}

public class MouseClick {
    public static void main(String[] args) {
        JFrame f = new JFrame("Mouse Click");
        ClickPanel p = new ClickPanel();
        f.add(p);
        f.setSize(300, 200);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        p.requestFocusInWindow();
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

1. Five methods, one used — MouseAdapter hides the four we do not need. mousePressed fires when the button goes down; mouseClicked only after a press and release without moving.

2. Static helper — SwingUtilities.isRightMouseButton(e) is a static method: no object needed.

3. State — x, y, color and clicks are private fields of the panel.

Drag and Drop a Rectangle

Scenario: press on the box to pick it up, drag to move it, release to drop it. Three events, one story.

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class DragPanel extends JPanel {
    private final Rectangle box = new Rectangle(60, 50, 80, 50);
    private int offX, offY;
    private boolean dragging = false;

    DragPanel() {
        MouseAdapter ma = new MouseAdapter() {
            @Override
            public void mousePressed(MouseEvent e) {
                if (box.contains(e.getPoint())) {
                    dragging = true;
                    offX = e.getX() - box.x;
                    offY = e.getY() - box.y;
                    repaint();
                }
            }
            @Override
            public void mouseDragged(MouseEvent e) {
                if (dragging) {
                    box.setLocation(e.getX() - offX, e.getY() - offY);
                    repaint();
                }
            }
            @Override
            public void mouseReleased(MouseEvent e) {
                dragging = false;
                repaint();
            }
        };
        addMouseListener(ma);
        addMouseMotionListener(ma);
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(dragging ? Color.RED : Color.ORANGE);
        g.fillRect(box.x, box.y, box.width, box.height);
    }
}

public class MouseDrag {
    public static void main(String[] args) {
        JFrame f = new JFrame("Drag a Rectangle");
        DragPanel p = new DragPanel();
        f.add(p);
        f.setSize(360, 240);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        p.requestFocusInWindow();
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

1. One adapter, two interfaces — MouseAdapter implements MouseListener and MouseMotionListener, so one object is registered twice.

2. Offset — offX and offY remember where inside the box we grabbed it, so it does not jump to the pointer.

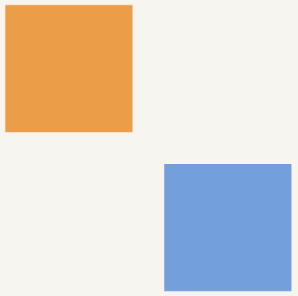
3. State machine — dragging is false, then true between press and release. mouseDragged does nothing unless it is true.

Collision: Do Two Rectangles Overlap?

A collision test is a yes or no question about two boxes.



overlap: intersects = true



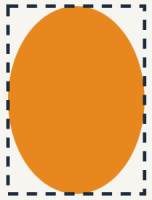
apart: intersects = false

They overlap only if they overlap **horizontally and vertically**.

java.awt.Rectangle already knows this: a.intersects(b).

We use boxes, not exact shapes. For eggs and baskets that is good enough.

Collision Details: Boxes, Edges and Speed



Egg: an oval



Basket: a curve

collision uses the boxes

Bounding box. We test the rectangles that surround the shapes, not the shapes. A corner of the egg's box can overlap while the oval itself does not touch: close enough for a game.

Edges. Rectangles that only share an edge do not intersect. intersects() needs a positive overlap area.

Speed. A very fast object can jump over a thin one between two ticks. Our egg moves 4 pixels per tick against a 30-pixel basket, so it is safe.

Predict first: if the egg moved 80 pixels per tick, could it pass through the basket without a collision?

Hit Test with Rectangle.intersects()

Scenario: a green box follows the mouse and turns red while it overlaps the gray target.

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class HitPanel extends JPanel {
    private final Rectangle target = new Rectangle(120, 70, 100, 60);
    private final Rectangle mover = new Rectangle(0, 0, 50, 40);

    HitPanel() {
        addMouseMotionListener(new MouseMotionAdapter() {
            @Override
            public void mouseMoved(MouseEvent e) {
                mover.setLocation(e.getX() - 25, e.getY() - 20);
                repaint();
            }
        });
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.GRAY);
        g.fillRect(target.x, target.y, target.width, target.height);
        g.setColor(mover.intersects(target) ? Color.RED : Color.GREEN);
        g.fillRect(mover.x, mover.y, mover.width, mover.height);
    }
}

public class HitDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Hit Test");
        HitPanel p = new HitPanel();
        f.add(p);
        f.setSize(340, 220);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        p.requestFocusInWindow();
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

1. A ready-made class — Rectangle has x, y, width, height and the methods intersects() and setLocation().

2. One boolean decides — the ternary picks the color; no manual coordinate arithmetic.

3. Reuse — next we ask the egg and the basket for their Rectangles.

Predict first: If two boxes only touch along an edge, does intersects() return true or false?

OOP 5: Each Object Reports Its Own Box

The panel does not know how wide an egg is. It asks. **Predict first:** if the egg became 40 pixels wide, which classes would have to change?

Egg

```
Rectangle getBounds() {
    return new Rectangle(x, y, 30, 40);
}
```

Basket

```
Rectangle getBounds() {
    return new Rectangle(x, y, W, H);
}
```

GamePanel

```
if (egg.getBounds()
        .intersects(basket.getBounds())) {
    score++;
}
```

If the egg gets bigger, only Egg changes. The panel code stays exactly as it is.

Encapsulation: x, y and sizes stay private. A new Rectangle is returned each time, so nobody can change the egg through it.

The Update Step: Move, Catch or Miss

Scenario: one tick of the game. Input applied, egg moved, then exactly one of: caught, missed, or nothing.

FRAGMENT: A METHOD OF GamePanel

```
private void update() {
    if (left)  basket.moveLeft();
    if (right) basket.moveRight();
    basket.clamp(getWidth());

    boolean wrapped = egg.fall(getHeight());
    if (egg.getBounds().intersects(basket.getBounds())) {
        score++;
        respawn();
    } else if (wrapped) {
        missed++;
        respawn();
    }
    repaint();
}

private void respawn() {
    egg.respawn(rnd.nextInt(getWidth() - 30));
}
```

OUTPUT

egg and basket…	result
overlap	score + 1, new egg
egg left the field	missed + 1, new egg
neither	keep falling

WHAT TO NOTICE

1. Order — input first, then movement, then the rules, then repaint.

2. else if — a tick can end in at most one outcome.

3. respawn() — a private helper; both branches share it instead of repeating code.

Predict first: What happens to the score if we delete respawn() after a catch?

The "Score Jumps by 17" Bug

Predict first: what happens to the score if the egg is not moved away after a catch? Here is the answer. A caught egg overlaps the basket for many ticks in a row. If we only add 1 when they overlap, we add 1 on every tick.

Without respawn()

tick	overlap?	score
1	yes	1
2	yes	2
3	yes	3
...	...	...
17	yes	17

With respawn() on the catch

tick	overlap?	score
1	yes: count, egg back to top	1
2	no (egg is at the top)	1
3	no	1

Fix: the catch consumes the egg. Move it away in the same tick you count it.

The Timer Across Three Weeks

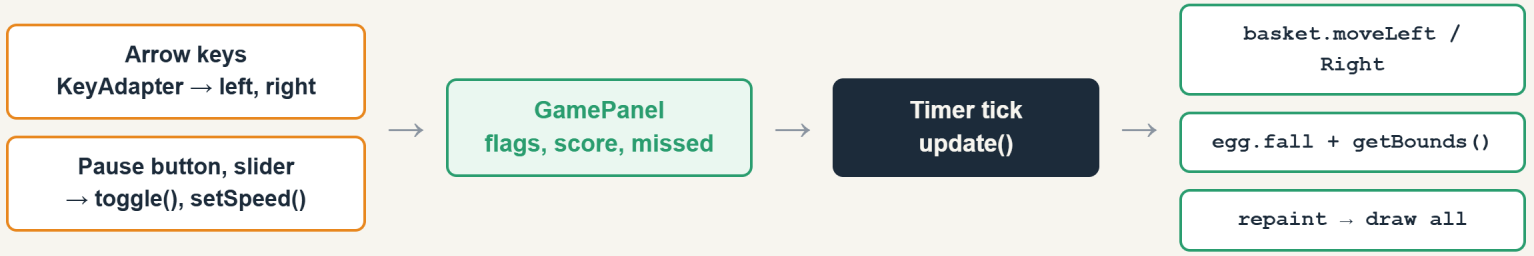
The line new Timer(16, e -> ...) never changed. Its listener grew.

	Week 3	Week 4	Week 5
The listener	moves a component	updates numbers, calls repaint()	applies input, moves objects, checks catches, updates score, calls repaint()
Who changes the state	the Timer	the Timer	the Timer; in a game panel the key and mouse listeners only record input
Who draws	JLabel components	paintComponent	paintComponent
Other event sources	button, slider	button, slider	keys, mouse, button, slider

In a game panel: one writer for the game state, and listeners that only leave notes. That keeps a growing game predictable. (In the small demos, listeners change their own square directly, and that is fine.)

Falling Egg v2: How the Pieces Connect

v1 plus a player, a basket and a score.



Components used: JPanel subclass, KeyAdapter, Timer, JButton, JSlider, Rectangle. Nothing new except how they are wired.

Falling Egg v2, Part 1: The Egg

The Egg gains respawn() and getBounds(). Basket was on an earlier slide; both go in the file.

```
// Basket class from the earlier slide goes in this file too
class Egg {
    private int x, y;
    private final int speed;

    Egg(int x, int speed) { this.x = x; this.speed = speed; }

    boolean fall(int fieldHeight) {
        y += speed;
        if (y > fieldHeight) { y = 0; return true; }
        return false;
    }

    void respawn(int newX) { x = newX; y = 0; }

    Rectangle getBounds() { return new Rectangle(x, y, 30, 40); }

    void draw(Graphics g) {
        g.setColor(Color.ORANGE);
        g.fillOval(x, y, 30, 40);
        g.setColor(Color.WHITE);
        g.fillOval(x + 7, y + 8, 8, 10);
    }
}
```

OUTPUT



WHAT TO NOTICE

1. Why respawn(newX)? — the panel decides where; the egg moves itself there and back to the top.

2. getBounds() — returns a fresh Rectangle, so nobody can edit the egg's private fields through it.

3. Still no Swing component — Egg only needs a Graphics.

Falling Egg v2, Part 2: The GamePanel (Setup and Controls)

The panel owns the egg, the basket, the Timer and the input flags. Its public doors are toggle() and setSpeed().

CONTINUES ON THE NEXT SLIDE (SAME CLASS)

```
class GamePanel extends JPanel {
    private final Egg egg = new Egg(100, 4);
    private final Basket basket = new Basket(150, 320);
    private final java.util.Random rnd = new java.util.Random();
    private final Timer timer;
    private boolean left, right;
    private int score = 0, missed = 0;

    GamePanel() {
        setPreferredSize(new Dimension(400, 360));
        setBackground(new Color(222, 240, 255));
        setFocusable(true);
        addKeyListener(new KeyAdapter() {
            @Override public void keyPressed(KeyEvent e)  { setKey(e.getKeyCode(), true); }
            @Override public void keyReleased(KeyEvent e) { setKey(e.getKeyCode(), false); }
        });
        addFocusListener(new FocusAdapter() {
            @Override public void focusLost(FocusEvent e) { left = false; right = false; }
        });
        timer = new Timer(16, e -> update());
        timer.start();
    }

    private void setKey(int code, boolean down) {
        if (code == KeyEvent.VK_LEFT)  left = down;
        if (code == KeyEvent.VK_RIGHT) right = down;
    }

    boolean toggle() {
        if (timer.isRunning()) timer.stop(); else timer.start();
        return timer.isRunning();
    }

    void setSpeed(int delay) { timer.setDelay(delay); }
```

OUTPUT



WHAT TO NOTICE

1. **Inheritance + composition —** GamePanel IS a JPanel and HAS an Egg, a Basket and a Timer.
2. **Input is only recorded —** the key listeners set left and right; update() will use them.
3. **Encapsulation —** timer, flags and score are private.

Falling Egg v2, Part 3: The GamePanel (Game Logic and Drawing)

The second half of GamePanel: one update step and one render step.

SECOND HALF OF GamePanel

```
    private void update() {
        if (left)  basket.moveLeft();
        if (right) basket.moveRight();
        basket.clamp(getWidth());

        boolean wrapped = egg.fall(getHeight());
        if (egg.getBounds().intersects(basket.getBounds())) {
            score++;
            respawn();
        } else if (wrapped) {
            missed++;
            respawn();
        }
        repaint();
    }

    private void respawn() {
        egg.respawn(rnd.nextInt(getWidth() - 30));
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        egg.draw(g);
        basket.draw(g);
        g.setColor(Color.DARK_GRAY);
        g.drawString("Score: " + score + " Missed: " + missed, 10, 20);
    }
}
```

OUTPUT



WHAT TO NOTICE

1. **update() —** input, movement, rules, repaint: the order from the Week 5 pattern.
2. **paintComponent —** reads state and draws it; it changes nothing.
3. **Delegation —** each object draws itself; the panel only gives them its Graphics.

Falling Egg v2, Part 4: The Window

Imports: javax.swing., java.awt.*, java.awt.event.*. The full file is in the lab notes.*

```
public class FallingEggV2 {
    public static void main(String[] args) {
        JFrame f = new JFrame("Falling Egg v2");
        GamePanel game = new GamePanel();
        JButton toggle = new JButton("Pause");
        JSlider speed = new JSlider(5, 40, 16);
        toggle.setFocusable(false);
        speed.setFocusable(false);

        JPanel bar = new JPanel();
        bar.add(toggle);
        bar.add(new JLabel("Tick delay:"));
        bar.add(speed);

        toggle.addActionListener(e ->
            toggle.setText(game.toggle() ? "Pause" : "Start"));
        speed.addChangeListener(e -> game.setSpeed(speed.getValue()));

        f.add(game);
        f.add(bar, BorderLayout.SOUTH);
        f.pack();
        f.setResizable(false);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        game.requestFocusInWindow();
    }
}
```

OUTPUT



WHAT TO NOTICE

1. The focus fix is in use — button and slider are not focusable, so the arrow keys keep working after you click them.

2. A clean boundary — main only knows toggle() and setSpeed().

3. setResizable(false) — the basket and field have fixed pixel positions, so we keep the size fixed.

Predict first: Does the Delay slider change only the egg? What else becomes faster or slower?

What Does the Speed Slider Really Change?

We call it a speed slider, but it changes the Timer's delay. One Timer drives everything.

timer.setDelay(ms)

```
speed.addChangeListener(e ->
    game.setSpeed(speed.getValue()));
```

- the egg falls faster or slower
- the basket moves faster or slower
- the whole game updates more or less often

egg.setSpeed(pixels)

```
private int speed;
void setSpeed(int s) { speed = s; }
```

- only the egg changes
- the basket keeps its speed
- the Timer beat stays constant

Predict first: which design would you choose for "the eggs get faster as your score grows"? Week 6 uses the second one, because eggs will have their own speeds.

Common Mistakes I: The Keys Do Nothing

The panel never got the focus

Symptom: no key reaches keyPressed.

Fix: setFocusable(true), and requestFocusInWindow() after setVisible(true).

A clicked button stole the focus

Symptom: the keys work until you click Pause, then stop.

Fix: setFocusable(false) on buttons and sliders, or use key bindings.

The listener was added to the frame

Symptom: nothing happens, or only sometimes.

Fix: add it to the same component that has the focus, normally the game panel.

Common Mistakes II: Listener Problems

A lambda for KeyListener or MouseListener

Symptom: compile error, "not a functional interface".

Fix: use new KeyAdapter() { ... } or new MouseAdapter() { ... } and override what you need.

No keyReleased

Symptom: the basket keeps sliding after you let go.

Fix: clear the flag in keyReleased.

Moving the basket inside keyPressed

Symptom: jerky motion with a pause after the first step.

Fix: set a flag in keyPressed; move on the Timer beat.

Common Mistakes III: Collision and Coordinates

Counting a catch on every tick

Symptom: the score jumps by 15 to 20 for one egg.

Fix: respawn the egg in the same tick you count the catch.

Forgetting clamp()

Symptom: the basket leaves the field on either side.

Fix: clamp x into [0, width - basketWidth] after every move.

Mouse coordinates from the wrong component

Symptom: the circle is offset from the pointer.

Fix: e.getX() is relative to the component that has the listener: attach it to the panel you draw on.

Lab Goals I: Keyboard and Focus (Easy to Medium)

- 1. Key echo: show the last key pressed
- 2. Move a square with the arrow keys (jerky)
- 3. Reproduce and fix the focus problem
- 4. Smooth movement with flags and a Timer
- 5. The same square with key bindings
- 6. Mouse follow: a circle under the pointer

Lab Goals II: Mouse, Collision and the Game (Medium to Hard)

- 7. Mouse clicks: left and right button, a counter
- 8. Hit test with Rectangle.intersects()
- 9. The Basket class, moved with the arrow keys
- 10. Catch the egg: collision, score and missed
- 11. Falling Egg v2: Pause, speed slider and the focus fix
- 12. Challenge: keyboard or mouse control, chosen with a JComboBox

NEXT WEEK

Week 6: Many Eggs with ArrayList

One egg at a time is not much of a game. Next week eggs spawn over time and live in an ArrayList. We loop with for-each and Iterator, remove caught or missed eggs safely, and add lives and levels.

Questions?